

*** What is the difference between **Comparable<T>** and **Comparator<T>** interface.

Comparable<T>	Comparator<T>
1) Available in java.lang package.	1) Available in java.util package.
2) public int compareTo(T x) [One Parameter]	2) public int compare(T x, T y) [Two Parameters]
3) We can write only one sorting logic.	3) We can write multiple sorting logic.
4) In order to write sorting logic, We need to modify the BLC class due to current object support (this keyword)	4) We need not to modify the BLC class.
5) It provides default natural sorting order.	5) It provides user-defined sorting order.
6) Arrays.sort(Object []jarr)	6) Arrays.sort(Object []jarr, Comparator<T> cmp)

- * Comparable<T> and Comparator<T> both are the functional interfaces.
- * Both are used to sort the **custom objects** based on some comparison.
- * In order to sort the custom objects, We should use Arrays.sort() method.
- * sort() method internally using compareTo(T x) method.
- * We can only say two objects are comparable objects, if and only if the corresponding class implements from Comparable<T> interface.
- * If a class does not implement from Comparable<T> (then It is a not a Comparable object) and if we try to compare by using Arrays.sort() method then it will generate [java.lang.ClassCastException](#)
- * All the Wrapper classes & String class implements from Comparable<T> so comparable types of objects

```
//Program on Comparable<T> :
-----
package com.ravi.comparable;

import java.util.Arrays;

record Employee(Integer id, String name) implements Comparable<Employee>
{
    @Override
    public int compareTo(Employee e2)
    {
        return Integer.compare(this.id, e2.id);
    }
}

public class EmployeeComparable
{
    public static void main(String[] args)
    {
        Employee []employees = new Employee[3];

        employees[0] = new Employee(222, "Zaheer");
        employees[1] = new Employee(333, "Aryan");
        employees[2] = new Employee(111, "Raj");

        IO.println("Before Sorting :");
        for(Employee employee : employees)
        {
            IO.println(employee);
        }

        Arrays.sort(employees);

        IO.println("After Sorting based on the ID :");
        for(Employee employee : employees)
        {
            IO.println(employee);
        }
    }
}
```

Limitation Of Comparable<T>

* While working with Comparable<T>, We need to modify the source code because this keyword (current object support is required), if the BLC class is provided by some 3rd party in jar file format then Comparable<T> will not work (Source code modification is compulsory)

* compareTo(T x) method accepts one parameter so, to compare two objects, this keyword support is required hence we cannot write Lambda expression for Comparable<T>

Note : In order to solve the above said issues Comparator<T> came into the picture

```
//Program on Comparator<T>
-----
package com.ravi.comparator;

import java.util.Arrays;
import java.util.Comparator;

record Product(Integer id, String name, Double price)
{
}

public class ProductComparator
{
    public static void main(String[] args)
    {
        Product []products = new Product[3];

        products[0] = new Product(333, "Camera", 98000D);
        products[1] = new Product(222, "Laptop", 24000D);
        products[2] = new Product(111, "Mobile", 54000D);

        IO.println("Original Data :");

        for(Product product : products)
        {
            IO.println(product);
        }

        IO.println("Sorting based on the ID :");

        //Anonymous inner class
        Comparator<Product> cmpId = new Comparator<Product>()
        {
            @Override
            public int compare(Product p1, Product p2)
            {
                return Integer.compare(p1.id(), p2.id());
            }
        };

        Arrays.sort(products, cmpId); //compare() method

        for(Product product : products)
        {
            IO.println(product);
        }

        IO.println("Sorting based on the Name :");
        Comparator<Product> cmpName = (p1, p2)-> p1.name().compareTo(p2.name());
        Arrays.sort(products, cmpName);

        for(Product product : products)
        {
            IO.println(product);
        }

        IO.println("Sorting based on the Price :");

        Arrays.sort(products, (p1,p2)-> Double.compare(p1.price(), p2.price()));

        for(Product product : products)
        {
            IO.println(product);
        }
    }
}
```

```
Another Program on Comparator<T>
-----
package com.ravi.comparator;

import java.util.Arrays;

public class ComparatorDemo
{
    public static void main(String[] args)
    {
        Integer numbers[] = {90,56,12,45,34};
        Arrays.sort(numbers,(i1, i2)-> i2.compareTo(i1));
        IO.println(Arrays.toString(numbers));

        String []fruits = {"Orange", "Mango", "Grapes", "Apple"};
        Arrays.sort(fruits,(s1,s2)-> s2.compareTo(s1));
        IO.println(Arrays.toString(fruits));
    }
}
```